UNIVERSITY OF CAMBRIDGE

DiS MPHIL

Reproducing a Tuning-Free Plug-and-Play algorithm for Sparse-View CT

Research project report presented by

Emilia Zabrzanska

Department
Department of Physics (Cavendish Laboratory)

Degree
Master of Philosophy in Data Intensive Science

Magdalene College
University of Cambridge

July 2026

Word count: **[6974]**

(including tables, figure captions, and appendices, but excluding references and the auto-generation usage declaration)

Submitted in partial fulfillment of the requirements for the
Master of Philosophy in Data Intensive Science

Declaration

I, Emilia Zabrzanska of Magdalene College, being a candidate for the Master of Philosophy in Data Intensive Science, declare that this report and the work described in it are my own work, except where specified below and in the AI/LLM usage declaration (Section A.4).

This report has a word count of approximately **[6974]** words. In line with the course's word-count policy, the count includes the abstract, tables, figure and table captions and appendices, and excludes the references and the auto-generation usage declaration (Section A.4).

The reproduction of the Tuning-Free Plug-and-Play algorithm [Wei et al., 2022] was carried out within the LION toolbox, developed by the Cambridge Image Analysis group. A pull request contributing improvements to LION's FBPConvNet implementation was submitted as part of this work (PR #184 on the CambridgeCIA/LION repository).

Signed: Emilia Zabrzanska

Date: July 2026

Abstract

Sparse-view computed tomography (CT) reduces patient radiation dose by acquiring fewer projection angles, at the cost of creating a more ill-posed reconstruction problem. The Tuning-Free Plug-and-Play (TFPnP) method of Wei et al. [2022] addresses this by using reinforcement learning to select ADMM hyperparameters automatically. This project reproduces TFPnP within the LION toolbox, targeting 30-view parallel-beam CT on the LIDC-IDRI dataset with the natural-image DRUNet as a prior. The reproduction achieves 27.27 dB PSNR at 5% sinogram noise, an improvement of +2.96 dB over hand-calibrated Fixed PnP-ADMM, confirming the paper's central claim about the advantage of learned hyperparameter policies.

The evaluation also reveals a partial reversal of the paper's TFPnP/FBPConvNet baseline comparison, as with FBPConvNet trained on the target CT domain, it outperforms TFPnP by +1.31 dB at 5% noise, but TFPnP overtakes on all three metrics at 10% noise, revealing the noise-robustness advantage of the iterative plug-and-play architecture. An extension replacing the natural-image DRUNet with a CT-specific variant produces a +0.71 to +1.98 dB improvement in Fixed PnP-ADMM, growing with noise, but a full TFPnP policy retrain against the CT-DRUNet was destabilised by RL training difficulties that future work is needed to resolve. All divergences from the paper, including choices left underspecified, are consolidated in a single table with appropriate justifications. The reproduction is released as a working Python package integrated with LION, together with an upstream contribution fixing a bug in the toolbox's FBPConvNet baseline.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Computed tomography (CT) is one of the most widely used diagnostic imaging modalities in modern medicine, providing detailed three-dimensional views of internal anatomy. It is not, however, without cost, as CT scans use ionising X-ray radiation that can be harmful with repeated exposure. Reducing radiation dose while preserving diagnostic image quality is therefore a central concern in clinical practice. The most common strategies are low-dose CT and, the focus of this project, sparse-view CT, where fewer projection angles are acquired per scan to reduce total exposure. The resulting inverse problem is more ill-posed, since with fewer measurements, the reconstruction is under-determined, and standard analytical inversion produces prominent streak artefacts.

Classical sparse-view reconstruction relies on filtered backprojection (FBP) or iterative methods with hand-crafted regularisers such as total variation (TV). FBP is fast but produces the streak artefacts already mentioned, and TV mitigates these but imposes a piecewise-constant prior, penalising differences between neighbouring pixels. Subtle intensity variations are therefore treated the same as noise and get smoothed into uniform patches, losing fine anatomical detail. Learned methods now dominate reconstruction benchmarks, and fall into two main families. Two-step reconstruction methods such as FBPConvNet [Jin et al., 2017, Biguri et al., 2025] take the FBP reconstruction as input and train a network to refine it into a clean image. Plug-and-play (PnP) methods combine iterative optimisation frameworks such as ADMM with learned image denoisers, allowing a single denoiser to serve as a prior for any inverse problem. The trade-off is that the denoising strength σ and the penalty parameter μ must be tuned carefully per iteration and per image, or performance degrades quickly.

The Tuning-Free Plug-and-Play method (TFPnP) of Wei et al. [2022] addresses this by formulating the choice of σ and μ as a reinforcement learning problem, where a policy network is trained to select hyperparameters automatically at each ADMM iteration, based on the current state of the reconstruction. Once trained, the policy is able to operate without any per-problem tuning at inference time. Wei et al. [2022] report state-of-the-art results on several inverse imaging problems including sparse-view CT, where TFPnP outperforms both classical baselines and the FBPConvNet. Reproducing these results in an independent implementation, and testing their

sensitivity to methodological choices left underspecified in the paper, form the primary focus of this project.

This project reproduces TFPnP from scratch within the LION toolbox [Biguri et al., 2024], a Python framework for AI-driven inverse imaging developed at the University of Cambridge. The reproduction is evaluated on the LIDC-IDRI clinical lung CT dataset [Armato et al., 2011], using the DRUNet denoiser [Zhang et al., 2021]. Throughout the project, several implementation choices affecting the final result were identified and documented. Two extensions were also carried out, including a domain-adapted CT-DRUNet and a set of reward-shaping ablations to test alternative RL reward signals. The reproduction reveals both the practical strengths of TFPnP's approach, particularly its robustness across noise levels, as well as its sensitivities to hyperparameter and training-data choices that the original paper does not discuss, including how the choice of RL action ranges interacts with training stability, and how an in-domain denoiser affects reconstruction quality within a plug-and-play pipeline.

1.1 Contributions

The contributions of this project are as follows.

(i) A faithful reproduction of the TFPnP method within the LION toolbox, targeting 30-view parallel-beam sparse-view CT with the natural-image DRUNet as a prior.

(ii) A finding that domain-specific FBPConvNet training substantially changes the ranking between two-step reconstruction and plug-and-play methods relative to the paper's out-of-domain comparison, with FBPConvNet leading at 5% noise and TFPnP overtaking on all three metrics at 10% noise.

(iii) A CT-specific DRUNet variant that improves reconstruction quality by 0.71–1.98 dB PSNR over the natural-image DRUNet in Fixed PnP-ADMM, with the advantage growing as noise increases.

(iv) A reward-shaping ablation testing four alternative RL reward signals (SSIM, HaarPSI, and mixed rewards), showing that the paper's default PSNR reward remains a defensible choice.

(v) An upstream contribution to the LION toolbox in the form of pull request #184, fixing a bug in the FBPConvNet integration.

1.2 Report structure

Chapter 2 introduces the relevant technical background and positions TFPnP within the wider landscape of sparse-view CT reconstruction methods. Chapter 3 describes the reproduction's design and implementation, including all deviations from the paper. Chapter 4 presents the empirical evaluation, and Chapter 5 interprets the findings. Chapter 6 summarises the project and outlines directions for future work, and Chapter A provides supplementary material and an auto-generation usage declaration.

Chapter 2

Background and Related Work

This chapter provides the technical background needed to understand the reproduction described in Chapters 3–5, and positions TFPnP within the wider landscape of sparse-view CT reconstruction methods. It covers the reconstruction problem itself, classical and learned approaches, the ADMM algorithm and its plug-and-play extension, the DRUNet denoiser, and the reinforcement learning framework used by TFPnP to select ADMM hyperparameters automatically.

2.1 Sparse-view CT

CT reconstruction aims to recover an internal image x from a set of X-ray measurements y taken at different angles around the body, modelled by the forward equation

$$y = Ax + \epsilon \tag{2.1}$$

where A is the forward operator modelling the X-ray acquisition and ϵ is measurement noise, and the exact form of A depends on the scan geometry. For the parallel-beam geometry used in this project, A is the discrete Radon transform, mapping the image to its line integrals along parallel rays at each projection angle.

In a full-view acquisition, x can be recovered by inverting A analytically using filtered backprojection (FBP), but in sparse-view conditions, the sinogram is under-sampled along ray directions and produces streak artefacts in the FBP reconstruction, which are visible as diagonal or radial patterns aligned with the missing projection angles. Figure 2.1 illustrates the effect.

Because the inverse problem is under-determined, additional constraints or prior information about what images should look like are required to recover high-quality reconstructions. The choice of prior distinguishes the reconstruction methods reviewed in the rest of this chapter.

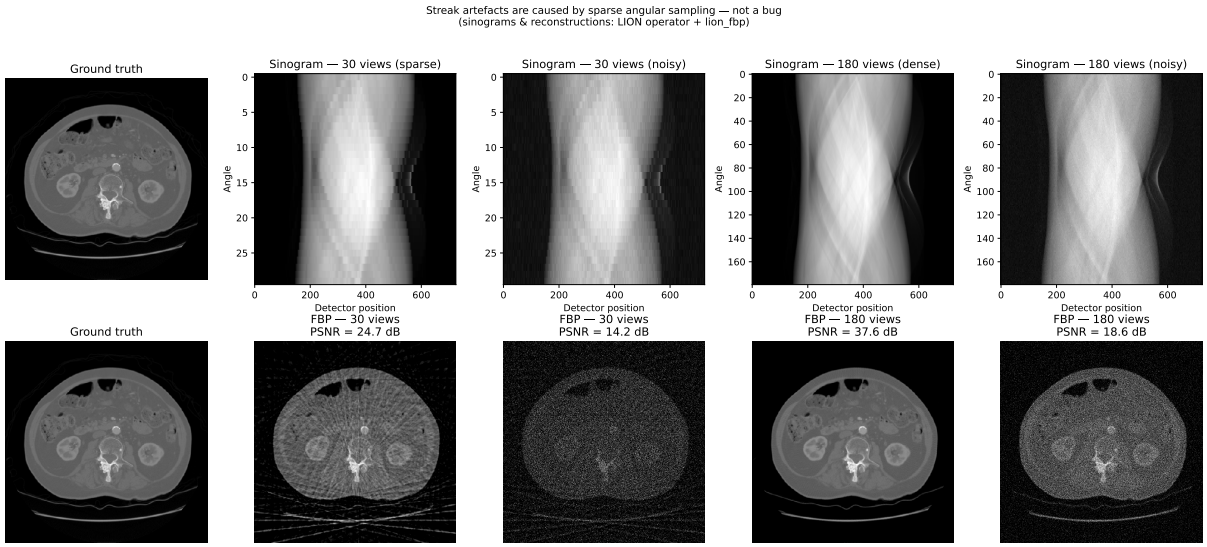

Figure 2.1: The sparse-view CT reconstruction problem, shown on noisy and clean images.

2.2 Classical reconstruction

Filtered backprojection (FBP) is the standard analytical method for CT reconstruction, applying a ramp filter in the frequency domain to compensate for the smoothing effect of backprojection. As introduced above, FBP produces streak artefacts under sparse-view conditions because the inversion problem becomes ill-posed.

Total variation (TV) regularisation is the standard iterative baseline for sparse-view reconstruction, formulating the inverse problem as the minimisation of a data-fidelity term together with a TV penalty encouraging piecewise-constant images. TV performs well on images with sharp boundaries and smooth regions, but its prior tends to produce cartoon-like reconstructions where fine anatomical detail is lost.

2.3 Two-step reconstruction methods

Two-step reconstruction methods [Biguri et al., 2025] take an analytical reconstruction such as FBP as input and train a neural network to refine it into a clean image. Their main advantage is inference speed, as a single forward pass produces a reconstruction almost instantly once the network is trained. The main disadvantage is that the trained network is specialised to the conditions seen during training, and may fail to generalise outside them.

FBPConvNet [Jin et al., 2017] is the method used in this project, and applies a U-Net to a sparse-view FBP reconstruction, mapping it to a clean full-view reconstruction. Taking the FBP reconstruction as input, rather than the raw sinogram, allows the network to operate in the image domain and avoids the need to learn the physics of the inverse problem.

2.4 Plug-and-play priors and ADMM

Plug-and-play (PnP) methods combine iterative optimisation with learned denoisers. Regularised reconstruction combines a data fidelity term measuring how well the reconstruction matches the measurements, and a prior term $R(x)$ encoding prior knowledge about what images should look like. This is known as the variational regularisation (VR) framework, or equivalently the maximum a posteriori (MAP) estimator under a Bayesian formulation.

$$x^* = \arg\min_x \frac{1}{2}\|Ax - y\|^2 + \lambda R(x) \tag{2.2}$$

The parameter λ balances the two objectives, and common choices for $R(x)$ include total variation, wavelet sparsity, and learned image priors.

The Alternating Direction Method of Multipliers (ADMM) is a standard algorithm for solving Equation (2.2), splitting the problem into three simpler sub-problems that are solved in turn, with an x-step applying the prior (a denoising operation), a z-step enforcing data fidelity, and a u-step tracking the disagreement between x and z across iterations. A penalty parameter μ controls how strongly x and z are coupled.

The plug-and-play insight is that the x-step reduces to a denoising problem, so rather than solving it exactly, it can be replaced by any denoiser D_σ, where σ controls the denoising strength. The denoiser then acts as an implicit prior, and no explicit form for $R(x)$ is required. The same denoiser can be reused across different inverse problems by pairing it with the appropriate data-fidelity term. The remaining difficulty is that both σ and μ must be chosen well, and typically require tuning per iteration and per image, which is the problem that TFPnP addresses.

2.5 DRUNet

DRUNet [Zhang et al., 2021] is a residual U-Net designed for plug-and-play priors, and the denoiser used in this project's TFPnP reproduction. It has 32.6 million parameters organised as a four-stage encoder-decoder with residual blocks and skip connections. Its input is the noisy image stacked with an additional channel holding a constant noise-level value σ, so a single trained network can denoise at any σ in its trained range by setting that channel accordingly. This makes DRUNet well suited to plug-and-play use, where σ typically changes across ADMM iterations. Figure 2.2 shows the architecture.

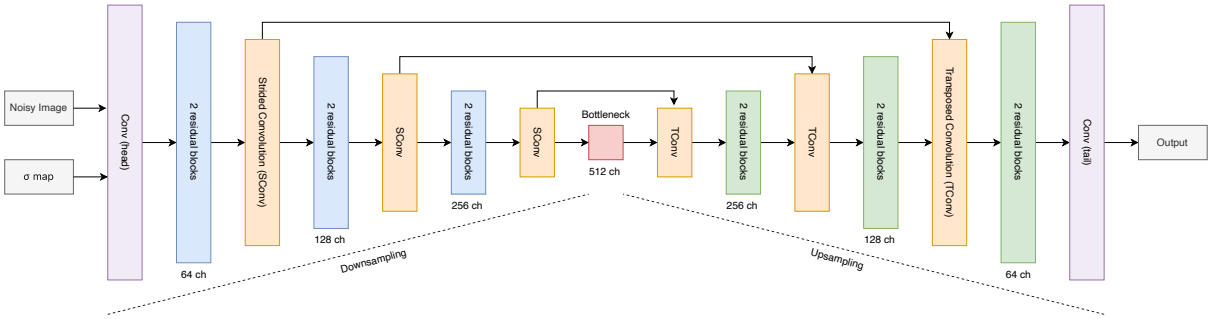

Figure 2.2: DRUNet architecture. The noisy image and constant-valued noise map σ are concatenated along the channel dimension and passed through a head convolution, three encoder stages of two residual blocks each, a four-block bottleneck, three decoder stages of two residual blocks each, and a tail convolution. Skip connections carry encoder features directly to the corresponding decoder stage. Adapted from Zhang et al. [2021].

DRUNet is trained on 400 natural images from BSD400, with additive Gaussian noise sampled uniformly from $\sigma \in [1, 50]$. The standard grayscale checkpoint (`drunet_gray.pth`) is the reference denoiser used by TFPnP for CT reconstruction and by many other plug-and-play works. Its use in this project is discussed in Section 3.3.

2.6 Tuning-free hyperparameter control with RL

TFPnP [Wei et al., 2022] treats the selection of σ and μ across ADMM iterations as a reinforcement learning problem. At each iteration, a policy network observes the current ADMM state (the variables x, z, u, noise level, and iteration progress) and outputs both a (σ, μ) prediction for the next m ADMM steps and a decision on whether to terminate. The policy is rewarded at each step by the PSNR gain minus a small constant penalty ($\eta = 0.05$), discouraging the policy from taking too many steps. The policy is trained once on a large image dataset, after which no further tuning is required at inference time.

Two sub-policies are also trained, with the discrete termination policy π_1 using REINFORCE, a policy-gradient method that reinforces actions leading to higher rewards, and the continuous parameter policy π_2 using deep deterministic policy gradient (DDPG), an actor-critic method backpropagated through the differentiable ADMM environment. Once trained, the policy produces σ and μ automatically.

A notable limitation of Wei et al. [2022]'s evaluation is that the policy and denoiser are both trained on natural images (PASCAL VOC and BSD400 respectively) and evaluated on medical images (CT slices from the COVID-19 CT dataset), making the entire pipeline out-of-domain relative to the target application. This project evaluates TFPnP against an in-domain FBP-ConvNet on the LIDC-IDRI dataset, and extends TFPnP with a CT-specific DRUNet to assess whether an in-domain denoiser improves reconstruction quality.

Chapter 3

Design and Implementation

3.1 Overview

The reproduction is implemented as a Python package, `ct_tfpnp` that integrates with the LION toolbox [Biguri et al., 2024] for the CT geometry, forward operator, and dataset handling. The package contains all components of the TFPnP algorithm (the ResNet-18 policy network, the residual value network, the ADMM environment, the DRUNet denoiser wrapper, and the RL training loop) as well as the baseline reconstruction methods (FBP, TV, DRUNet alone, FBP-ConvNet, and Fixed PnP-ADMM), and is released with a documented public API, a `pytest` suite, and Sphinx-generated documentation (Section A.3).

The full pipeline covers eleven trained TFPnP runs and two baseline model checkpoints (FBP-ConvNet and a CT-trained DRUNet), with each run producing diagnostic figures generated by `scripts/evaluate_run.py`. All experiments used the Cambridge Service for Data Driven Discovery (CSD3) HPC cluster (Chapter A).

3.2 Geometry, data, and noise conventions

All experiments use a sparse 30-view parallel-beam geometry, matching Wei et al. [2022]. As LION had no matching preset, a custom `Geometry` object was constructed with 30 angles uniformly spaced over $[0, \pi)$, a 512×512 image, and 724 detectors ($\lceil 512\sqrt{2} \rceil$, chosen to cover the image diagonal). The forward operator is LION's tomosipo-backed operator [Hendriksen et al., 2021], whereas Wei et al. [2022] use TorchRadon [Ronchetti, 2020]. The two are mathematically equivalent but differ in scaling conventions (Section 3.4).

Training and evaluation data come from LIDC-IDRI [Armato et al., 2011], which issplit at the patient level into 80/10/10 partitions using LION's default loader, providing approximately 3300, 401, and 389 slices in the training, validation, and test splits. Wei et al. [2022] evaluate on 50 slices from the COVID-19 CT dataset [Ma et al., 2021], so our test set is substantially larger. Image intensities are kept in linear attenuation coefficient (μ) units (approximately $[0, 2.5]\,\mathrm{cm}^{-1}$), the convention set by LION's `from_HU_to_mu` transform, rather than being rescaled to $[0, 1]$.

Sinogram noise follows Wei et al. [2022]'s percentage-of-signal convention, with additive Gaussian noise at 5%, 7.5%, or 10% of the clean sinogram's standard deviation and per-image seeds so paired comparisons use identical realisations. LION provides a more physically realistic Poisson-Gaussian noise model, but it takes physical parameters (photon count, detector gain) rather than a percentage fraction, so its levels do not map onto the paper's conditions and therefore the percentage convention is kept to keep the comparison close to the paper.

One consequence of retaining native μ units is that PSNR values here are approximately 5 dB higher than those in Wei et al. [2022], affecting cross-study comparison but not within-experiment rankings (Section 3.7, Section 5.4).

Figure 3.1: A lung slice from LIDC-IDRI, in native μ units (approximately $[0, 2.3]\,\mathrm{cm}^{-1}$).

3.3 TFPnP policy and RL training

The policy network follows Wei et al. [2022]. A ResNet-18 backbone extracts features from a five-channel input consisting of the current ADMM iterates (x, z, u), a noise-level map, and an iteration-progress map. Two heads share the resulting 512-dimensional feature vector, a two-way termination head predicting whether to stop, and a parameter head predicting the next $m = 5$ pairs of (σ, μ) via a sigmoid rescaled to the configured action ranges.

The backbone is torchvision's ResNet-18 with only the first convolution modified to accept five input channels. Two structural details differ from Table 1 of Wei et al. [2022], as their stem is 3×3 whereas torchvision's is 7×7, and our slices are 512×512 so layer4 outputs 16×16 rather than 4×4, but this is absorbed by an adaptive average pool. The value network also departs from the paper as Table 1 specifies the policy feature extractor with batch normalisation replaced by weight normalisation and ReLU by TReLU, whereas ours is eight residual blocks at width 64 with no normalisation and standard ReLU, preserving the removal of batch normalisation but not the replacements, with consequences analysed in Section 5.2.

The policy action ranges are $\sigma \in [1, 5]$ (calibrated to natural-image DRUNet's operating region (Section 4.4)) and $\mu \in [10, 100]$ (balances against the norm data-fidelity gradient (Section 3.4)). The RL formulation matches the paper's equations 15–17, with the value network trained on a temporal-difference loss ($\gamma = 0.99$) against a slowly-updated target network (EMA rate 10^{-3}). The termination policy π_1 uses REINFORCE with a value baseline, and the reward is $r = \Delta\text{PSNR} - \eta$ with $\eta = 0.05$. Learning rates are 10^{-4} (critic), 3×10^{-5} (shared features and termination head), and 10^{-6} (parameter head, kept low for the sensitivity of π_2's gradient through the ADMM rollout). The paper's $m = 5$, $N = 6$, $\eta = 0.05$ are preserved exactly.

3.4 ADMM step and z-step gradient normalisation

The ADMM environment implements the paper's equations 7–9, with the x-step applying the DRUNet denoiser as an implicit prior, the z-step enforcing data fidelity, and the u-step updating the dual variable. The z-step is solved by six gradient descent iterations, allowing the model-based DDPG update for π_2 to backpropagate through it using automatic differentiation.

LION's adjoint $A^\top$ is the raw, unnormalised backprojection of tomosipo's discrete Radon transform, so $A^\top(Az)$ produces per-voxel values far larger than the input image. Without adjustment, the data-fidelity gradient $A^\top(Az - y)$ would dominate the proximal term $\mu(z - \text{target})$ for any reasonable μ, making the policy's μ predictions meaningless. The backprojected residual is therefore normalised to unit maximum before being combined with the proximal term

$$\text{grad} = \frac{A^\top r}{\|A^\top r\|_\infty + \epsilon} + \mu(z - \text{target}), \tag{3.1}$$

where $r = Az - y$, target $= x + u$, and $\epsilon = 10^{-8}$, and the step size is scaled as $\alpha/(1 + \mu)$ with $\alpha = 1$ to keep the update bounded as μ grows. Our absolute μ values therefore differ from the paper's in scale but preserve their role in the ADMM balance, reflected in the $\mu \in [10, 100]$ range. This normalisation keeps μ meaningful under LION's operator scaling.

3.5 Baselines

The reproduction is validated against five baselines. Three of the paper's baselines (RED-CNN, LPD, RPGD) are absent from LION or would require training beyond the project timeline, and were not included (Section 5.4).

FBP applies a Ram-Lak filter followed by backprojection through LION's operator, with the analytical normalisation $\pi/(2N)$ for $N = 30$, clamped to non-negative values.

TV reconstruction uses proximal gradient descent (200 iterations) with total variation at $\lambda = 0.01$ and non-negativity enforced each iteration.

DRUNet alone applies the natural-image DRUNet as a single pass over the FBP reconstruction at fixed $\sigma = 10$. It is not one of the paper's baselines, but provides a lower bound illustrating how much of TFPnP's performance comes from ADMM's iterative refinement rather from than

the denoiser itself.

FBPConvNet [Jin et al., 2017] is a five-stage U-Net trained on the LIDC training split (250 patients, 80 epochs) using LION's implementation, MSE loss, and Adam at a fixed 5% noise level. This departs from Wei et al. [2022]'s FBPconv, whose near-constant PSNRs across noise levels (23.00, 22.94, 22.86 dB) suggests mixed-noise training. Fixed 5% training therefore makes evaluation at 7.5% and 10% out-of-distribution, and is central to the reversal narrative discussed in Section 4.1.

Fixed PnP-ADMM uses the same ADMM environment as TFPnP with hand-calibrated $\sigma = 1.5$, $\mu = 20$, which were selected as they maximised PSNR on a preliminary sweep. Any improvement of TFPnP over it can be attributed to policy-based adaptation.

3.6 CT-specific DRUNet training

The headline TFPnP result (`run_04`) uses the natural-image DRUNet, matching Wei et al. [2022]. As an extension, a CT-specific DRUNet is trained to test if an in-domain denoiser improves reconstruction quality, using the same architecture and checkpoint format as the pretrained version [Zhang et al., 2021] to allow for a drop-in replacement. Training uses the LIDC training split (3300 slices) for 50 epochs with Adam at 10^{-4}, cosine annealing, and batch size 8, sampling a random noise level $\sigma \in [0, 50]$ per image following Zhang et al. [2021]. Loss is MSE in the per-image $[0, 1]$ space, and best-checkpoint selection uses validation PSNR at $\sigma = 15$.

An initial drop-in test of the CT-DRUNet within `run_04`'s pipeline revealed a calibration mismatch where PSNR collapsed to approximately 13 dB. A sensitivity sweep (Section 4.4) located the CT-DRUNet's optimum at $\sigma \approx 8$, and when each denoiser is compared at its own optimum the CT-DRUNet outperforms the natural-image variant at every noise level (Section 4.5). The interpretation of this shift is revisited in Section 5.5.

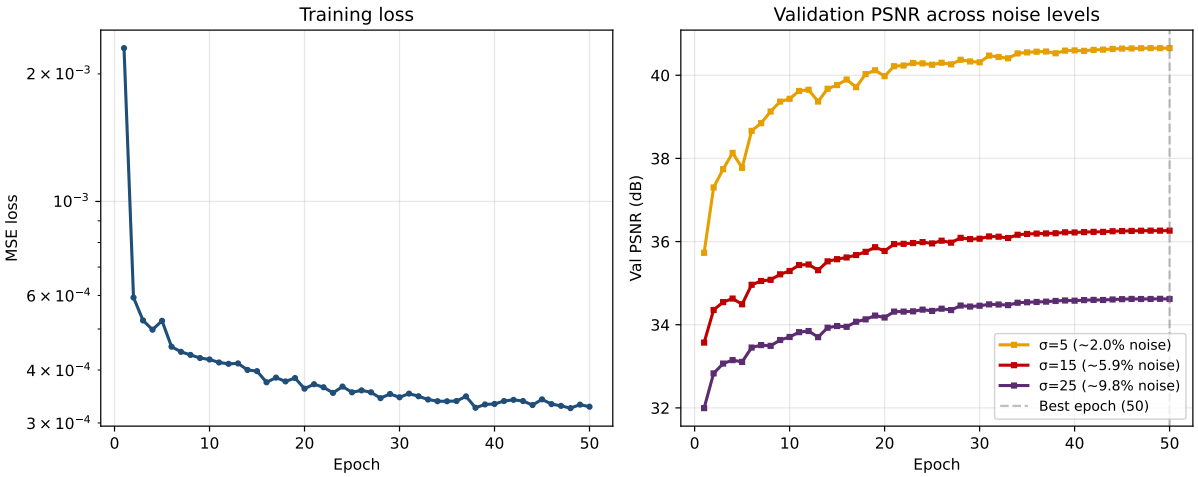

Figure 3.2: CT-DRUNet training curves. Left, MSE loss on the LIDC training split over 50 epochs. Right, validation PSNR at $\sigma \in \{5, 15, 25\}$. Best checkpoint by validation PSNR at $\sigma = 15$.

3.7 Evaluation metrics

Reconstruction quality is evaluated with PSNR, structural similarity (SSIM) [Wang et al., 2004], and Haar wavelet-based perceptual similarity (HaarPSI) [Reisenhofer et al., 2018]. PSNR is the paper's metric and is required for direct comparison, but the motivation for reporting SSIM and HaarPSI alongside it is discussed in Section 5.3. PSNR and SSIM are implemented in PyTorch to preserve differentiability through the reward chain (required for the model-based π_2 gradient), with both following LION's per-image convention `data_range`$= x_{\text{gt}}$.max(). HaarPSI is used only at evaluation and is loaded from the `piq` library [Kastryulin et al., 2022], which accepts a configurable `data_range` compatible with the μ-scaled inputs. All metrics are computed per-image and averaged over the test set.

Table 3.1: Implementation choices differing from Wei et al. [2022].

Aspect	Wei et al. [2022]	Ours	Reason
CT geometry	30-view TorchRadon	30-view custom LION `Geometry`	LION had no matching preset
Detector count	182	724 ($\lceil 512\sqrt{2} \rceil$)	Cover image diagonal
Forward projector	TorchRadon	LION and tomosipo	LION native
Policy backbone stem	3×3	7×7 (torchvision default)	Downsampling and stage widths unchanged, retraining was outside compute budget
Value network	Policy feature extractor, weight norm, TReLU	8 Residual blocks, unnormalised, ReLU	Simpler BN-free critic
Policy input	128×128	512×512	ADMM state at full resolution
Image intensity scale	[0, 1] normalised	Native μ	Preserve physical units
z-step gradient norm	None	Normalised $A^\top r$	LION operator scaling
PSNR data range	Not stated	Per-image x_{gt}.max()	Matches LION `data_range`
PSNR / SSIM backend	Not specified	Custom PyTorch reimplementations of LION reference	Differentiability for ADMM step
HaarPSI backend	Not used	`piq` library	Compatible with μ-unit inputs
Policy σ range	Not stated	$[1, 5]$	Natural DRUNet calibration
Policy μ range	Not stated	$[10, 100]$	Effective range post-gradient-normalisation
Denoiser training data	BSD400 natural images	Same (`run_04`), LIDC (`run_11`)	In-domain extension
Denoiser loss	L_1	MSE	PSNR-aligned
Denoiser batch size	32	8	Fits GPU budget
Policy learning rates	l_θ=10^{-4}, l_ϕ=5×10^{-5} (later halved)	Critic 10^{-4}, policy 3×10^{-5}, param 10^{-6}	Empirical stability with our action ranges
Policy training data	PASCAL VOC natural images	LIDC	In-domain policy training
FBPConvNet training noise	Mixed (implied by flat PSNR)	5% fixed	Simplest testable regime
Test set	COVID-19 lung, 50 images	LIDC-IDRI, 389 images	LION-native dataset

Chapter 4

Evaluation

4.1 Headline reproduction result

The headline TFPnP result comes from `run_04_pat_250_e80`, where the policy is trained on 250 LIDC patients over 80 epochs, using the natural-image DRUNet as the denoiser prior and PSNR as the reward function. All evaluations use the full LIDC-IDRI test set (389 images) at three sinogram noise levels (5%, 7.5%, 10%), and the full experiment configuration is documented in Section 3.3.

Six reconstruction methods are compared, including classical baselines (FBP and TV), the denoiser applied directly to the FBP reconstruction (DRUNet), a learned network (FBPConvNet), Fixed PnP-ADMM with hand-calibrated hyperparameters, and the TFPnP reproduction with the learned policy. Table 4.1 reports mean PSNR, SSIM, and HaarPSI at each noise level.

The reproduction confirms the paper's central claim, with TFPnP achieving **27.27 dB** PSNR at 5% noise, a +2.96 dB improvement over Fixed PnP-ADMM. Perceptual metrics (SSIM = 0.691, HaarPSI = 0.485) improve similarly over Fixed PnP-ADMM (0.499 and 0.405), confirming that the learned policy provides measurable value across all metrics.

One finding departs from the paper's ranking, as FBPConvNet reaches 28.58 dB at 5% noise, +1.31 dB above TFPnP, whereas Wei et al. [2022] report TFPnP ahead of FBPconv by +1.50 dB. The ranking inverts by 10% noise (Section 4.2), and the underlying causes are analysed in Section 5.1. Figure 4.1 shows a visual comparison of the best, median, and worst reconstructions.

Table 4.1: Reconstruction quality on the LIDC-IDRI test set (389 images).

	5% noise			**7.5% noise**			**10% noise**		
Method	PSNR	SSIM	HaarPSI	PSNR	SSIM	HaarPSI	PSNR	SSIM	HaarPSI
FBP	14.56	0.075	0.233	13.65	0.058	0.223	13.22	0.052	0.216
TV	19.67	0.288	0.390	18.35	0.271	0.348	17.50	0.248	0.315
DRUNet (σ=10)	21.80	0.455	0.384	19.50	0.418	0.312	18.16	0.391	0.245
FBPConvNet	**28.58**	**0.726**	**0.556**	**27.14**	0.619	**0.490**	24.73	0.463	0.385
Fixed PnP-ADMM	24.31	0.499	0.405	22.42	0.376	0.336	21.50	0.340	0.290
TFPnP (Ours)	27.27	0.691	0.485	26.58	**0.681**	0.443	**26.10**	**0.676**	**0.413**

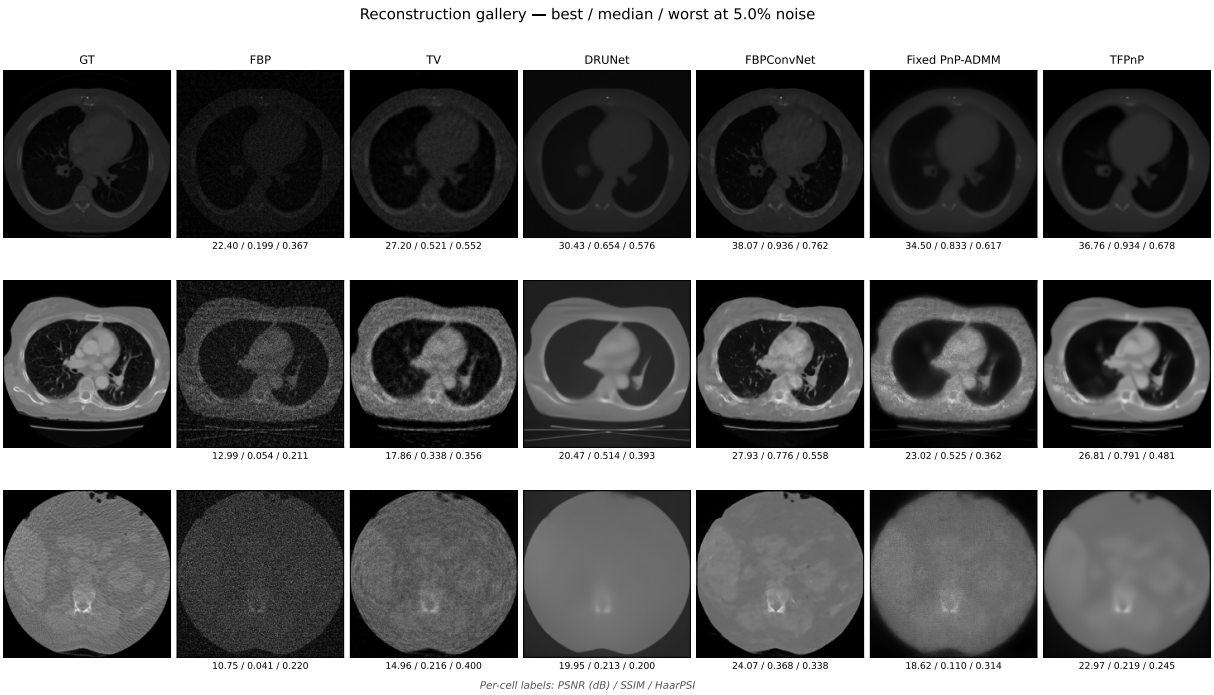

Figure 4.1: Reconstruction gallery for `run_04` at 5% noise. Rows show the best, median, and worst test images by TFPnP PSNR.

4.2 Cross-noise behaviour

TFPnP's practical value depends not only on its performance at the training noise level but also on how well it generalises across the noise range seen at inference. Figure 4.2 traces mean PSNR, SSIM, and HaarPSI as sinogram noise increases from 5% to 10%. TFPnP degrades gracefully, with PSNR dropping only -1.17 dB (from 27.27 to 26.10) and SSIM remains effectively flat (0.691 to 0.676). This robustness reflects the strength of the plug-and-play iterative approach, since the policy can adapt the denoising strength and penalty parameter to different noise levels.

By contrast, FBPConvNet degrades steeply across the same range (-3.85 dB PSNR, SSIM from 0.726 to 0.463), and TFPnP overtakes it on all three metrics at 10% noise. This is partly due to implementation differences, as our FBPConvNet was trained at fixed 5% noise (Section 3.5) and extrapolates at higher noise. However, the crossover at 10% suggests that the iterative structure of TFPnP provides a robustness that training-distribution matches alone would not explain, and is analysed further in Section 5.1.

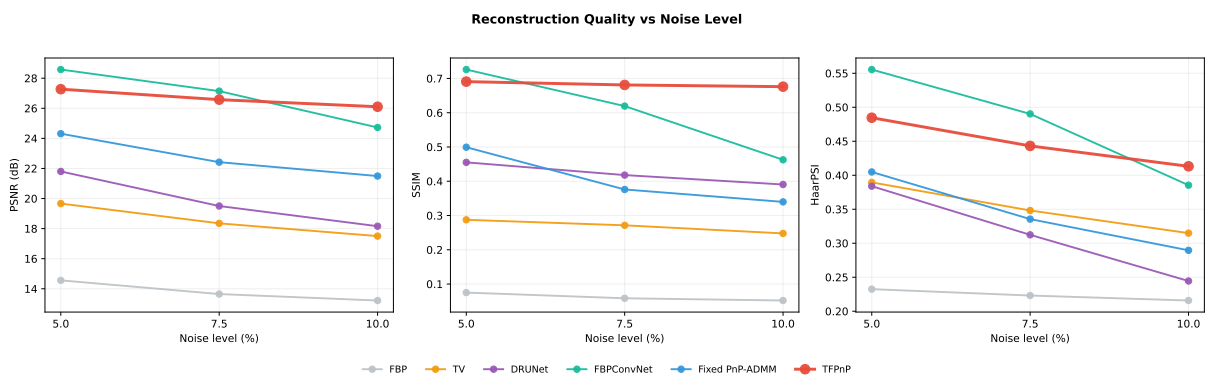

Figure 4.2: Mean PSNR, SSIM, and HaarPSI vs sinogram noise level.

4.3 Per-image variance

Averaging hides per-image variability, so Figure 4.3 separates this by showing TFPnP's three largest wins and losses over the next-best non-TFPnP method at 5% noise, ranked by per-image PSNR difference. TFPnP's wins tend to occur on homogeneous slices with large uniform regions, while losses occur on complex slices with high-frequency detail such as multiple small nodules or intricate anatomy that doesn't contain the lungs. Per-image variability means that no method dominates uniformly, and reconstructions of comparable quality across methods are common. This is consistent with the general finding that PnP and end-to-end methods have complementary strengths, with neither completely outperforming the other.

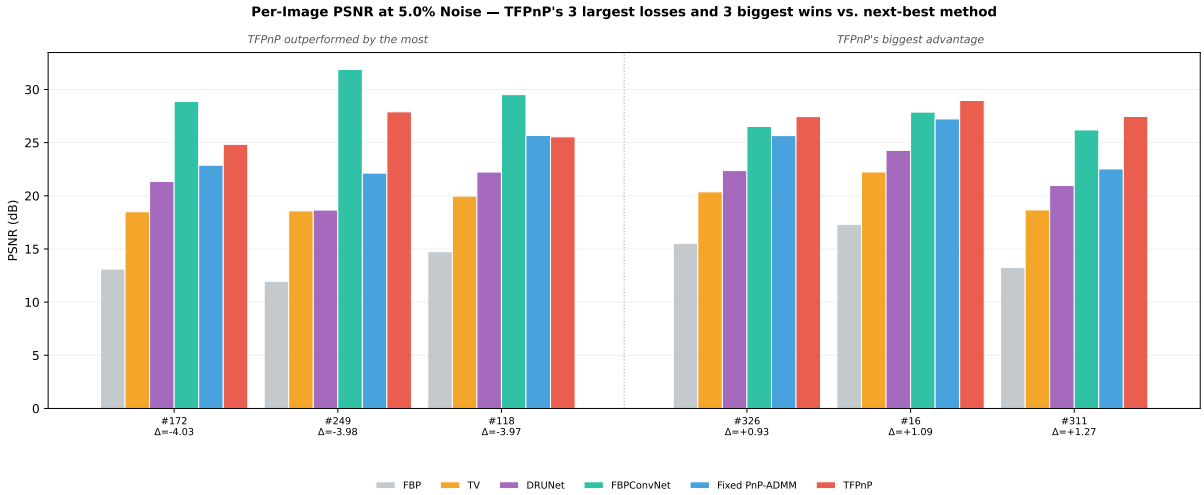

Figure 4.3: Per-image PSNR at 5% noise. TFPnP's three largest wins (left) and losses (right) versus the next-best non-TFPnP method.

4.4 Hyperparameter calibration sweeps

The Fixed PnP-ADMM baseline uses hand-calibrated $\sigma = 1.5$ and $\mu = 20$ values for the natural-image DRUNet, identified through sensitivity sweeps during notebook development. When the denoiser is swapped for the CT-trained DRUNet developed in Section 3.6, this calibration is no longer valid, and Figure 4.4 and Figure 4.5 present calibration sweeps for both denoisers. The CT-trained DRUNet peaks at $\sigma = 8.0$, and the interpretation of this shift is discussed in Section 5.5. The corresponding μ sweep at $\sigma = 8$ (see Figure 4.6) reveals a peak at $\mu = 10$, half the natural-DRUNet value.

Two limitations of these sweeps are worth noting. First, they were performed on a single slice at 5% noise, and a broader sweep across the validation set was intended as an extension but could not be completed due to computational constraints. Second, the sweep grids used $\sigma \in [1, 25]$ and $\mu \in [5, 50]$, but a finer grid around the peak could refine the calibration further.

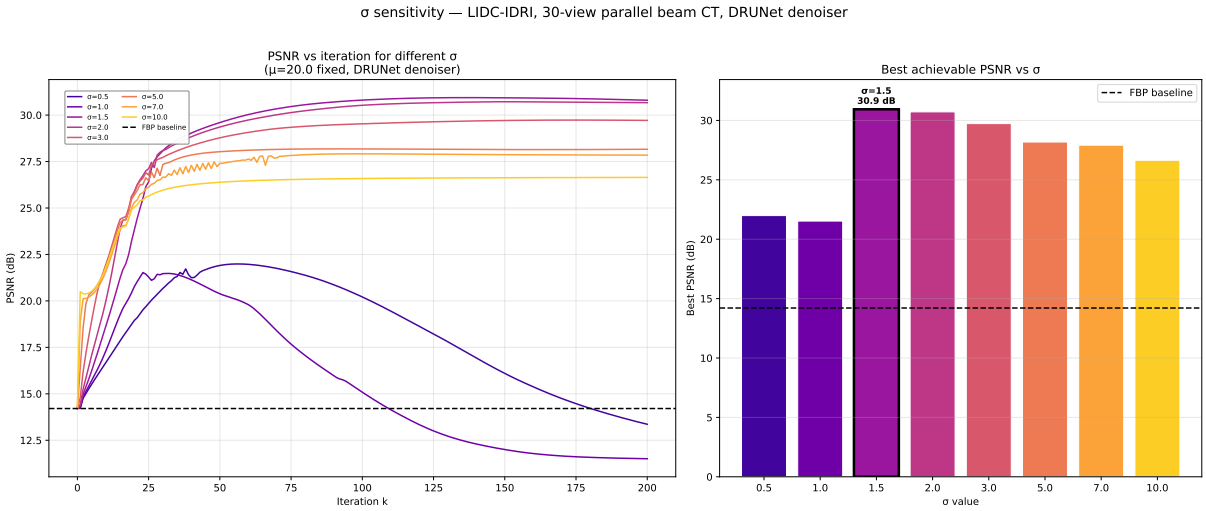

Figure 4.4: σ sensitivity for natural-image DRUNet within Fixed PnP-ADMM at 5% noise.

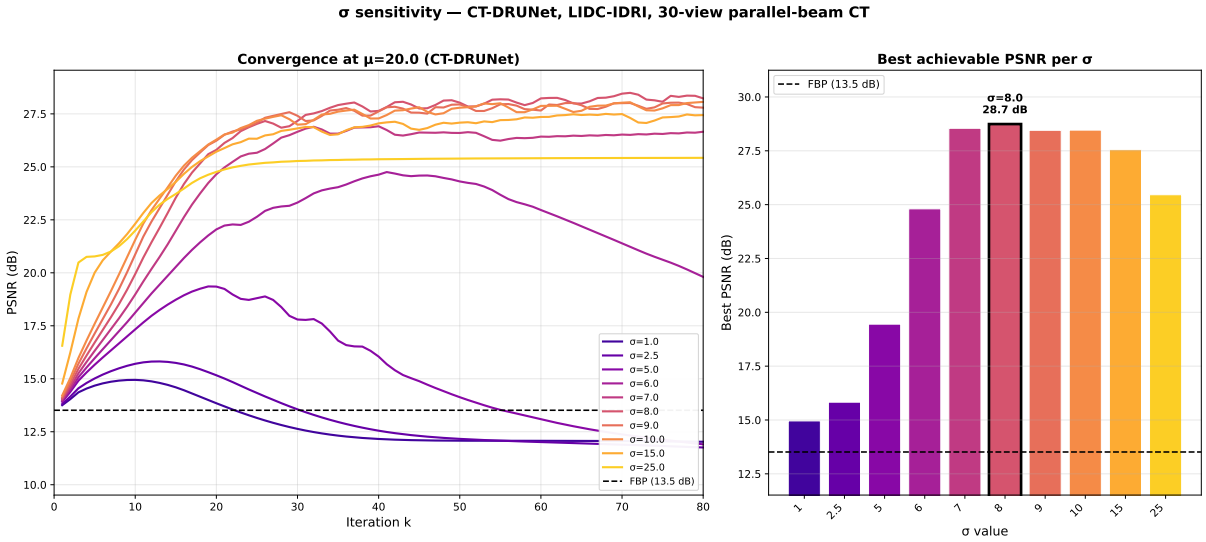

Figure 4.5: σ sensitivity for CT-trained DRUNet within Fixed PnP-ADMM at 5% noise.

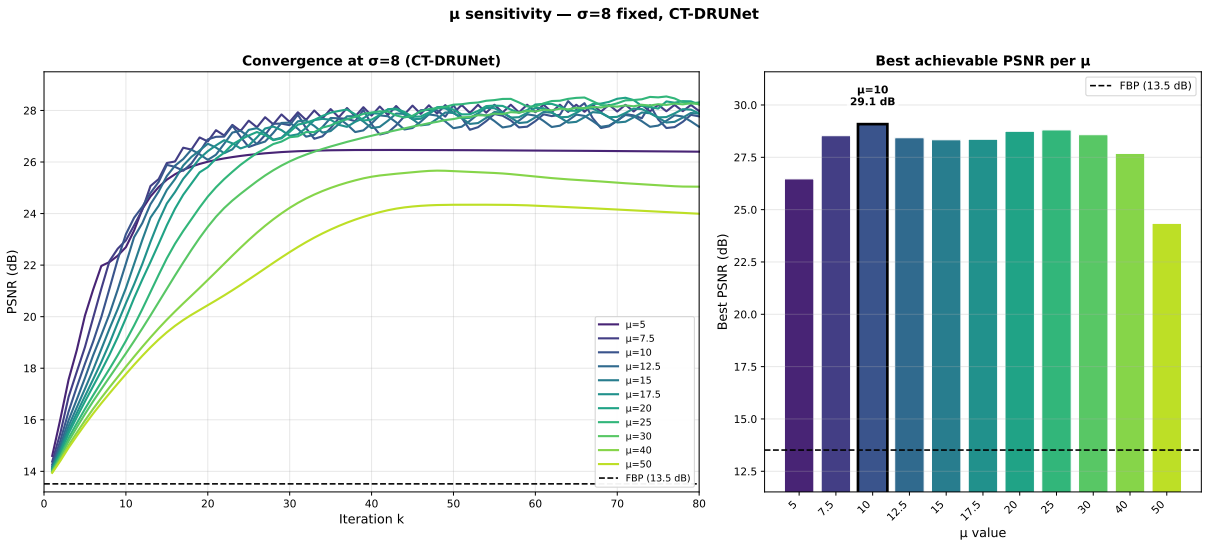

Figure 4.6: μ sensitivity for CT-trained DRUNet at $\sigma = 8$.

4.5 In-domain denoiser experiment

With the CT-trained DRUNet calibrated to its own operating point ($\sigma = 8$, $\mu = 10$), a paired comparison of Fixed PnP-ADMM using each denoiser at its respective optimum was performed on the full 389-image test set (Table 4.2). The CT-DRUNet outperforms the natural-image DRUNet at every noise level, and the advantage grows with noise, from +0.71 dB at 5% to +1.98 dB at 10%. SSIM improvements follow the same trend, from +0.037 to +0.090.

The advantage grows with noise and is discussed in Section 5.2. Figure 4.7 shows a visual comparison using Fixed PnP-ADMM at all three noise levels. TFPnP with the learned policy was not used, as retraining the policy against the CT denoiser (Section 4.6) encountered training instability that could not be resolved before a CSD3 cluster outage.

Table 4.2: Fixed PnP-ADMM at each denoiser's calibrated optimum.

	5% noise		**7.5% noise**		**10% noise**		**Advantage**	
Denoiser	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM
Natural DRUNet (σ=1.5)	28.06	0.752	26.52	0.695	25.41	0.652	–	–
CT DRUNet (σ=8)	**28.77**	**0.789**	**27.91**	**0.768**	**27.39**	**0.742**	+0.71 to +1.98	+0.037 to +0.090

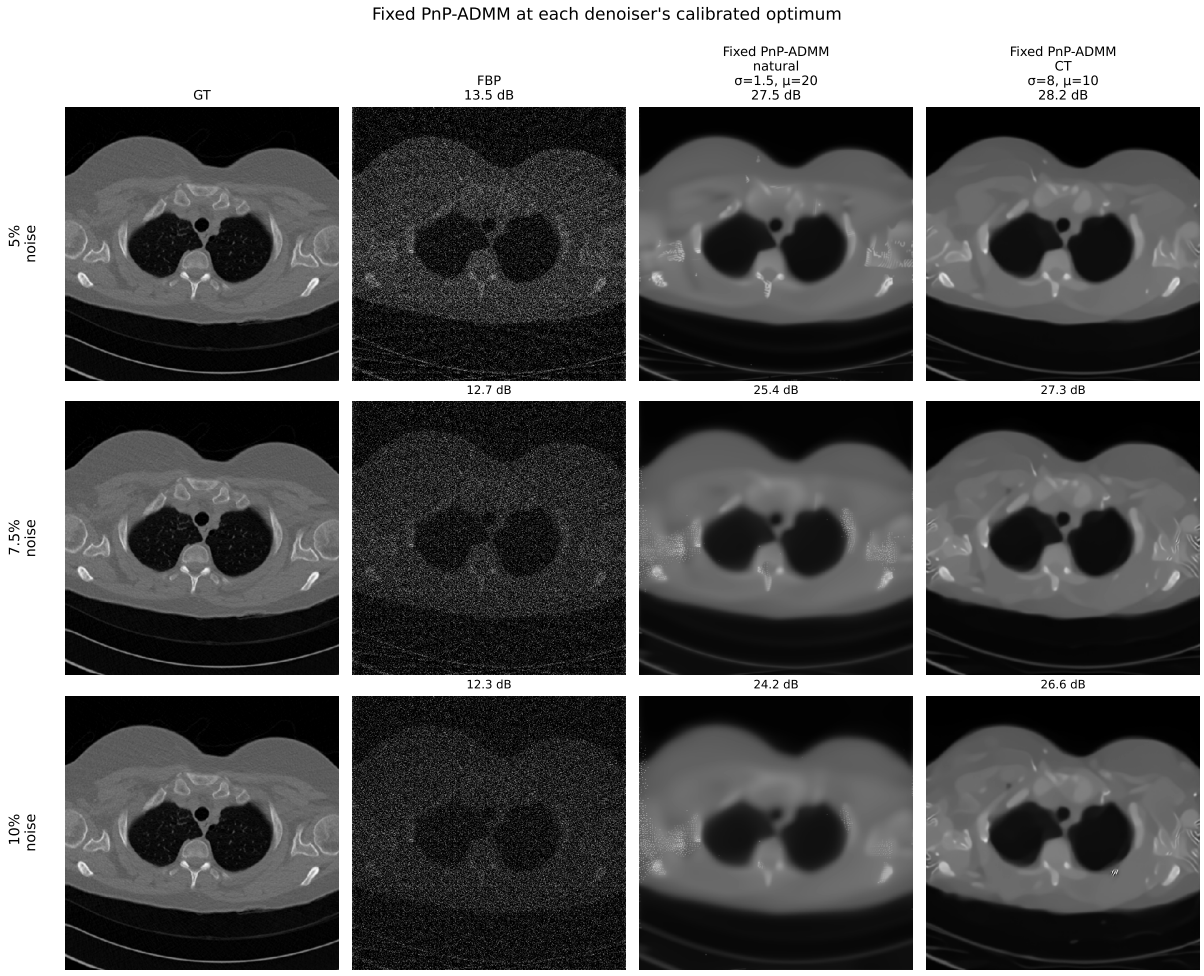

Figure 4.7: Fixed PnP-ADMM with denoisers at their calibrated optimum, at 5%, 7.5%, and 10% noise.

4.6 TFPnP policy retrain with CT-DRUNet

The natural next step is retraining the TFPnP policy against the CT-DRUNet, with a widened σ action range $[1, 15]$ to cover the new optimum and a narrowed μ range $[5, 50]$ chosen to keep the ADMM balance stable at the higher σ values. This run (`run_11`) underperformed `run_04`, reaching only 26.51 dB at 5% noise, well below `run_04`'s 27.27 dB and even the Fixed PnP-ADMM baseline with CT-DRUNet at 28.77 dB (Section 4.5).

Figure 4.8 illustrates the diagnostic training curves, showing the critic loss exploding to 10^{19} around policy update step 15,000 and the policy's mean σ output drifting from ~ 14 to ~ 9, with both training and validation PSNR collapsing from their earlier trajectory. The best checkpoint is saved from epoch 1, indicating no improvements were made. Since both the σ and μ ranges were changed simultaneously, the failure cannot be linked to either alone. Our value network also omits the weight normalisation and TReLU that Table 1 of Wei et al. [2022] specifies for the critic (Section 3.3), which could be another factor contributing to the failure of this run, as both are used for their stabilising effect on training.

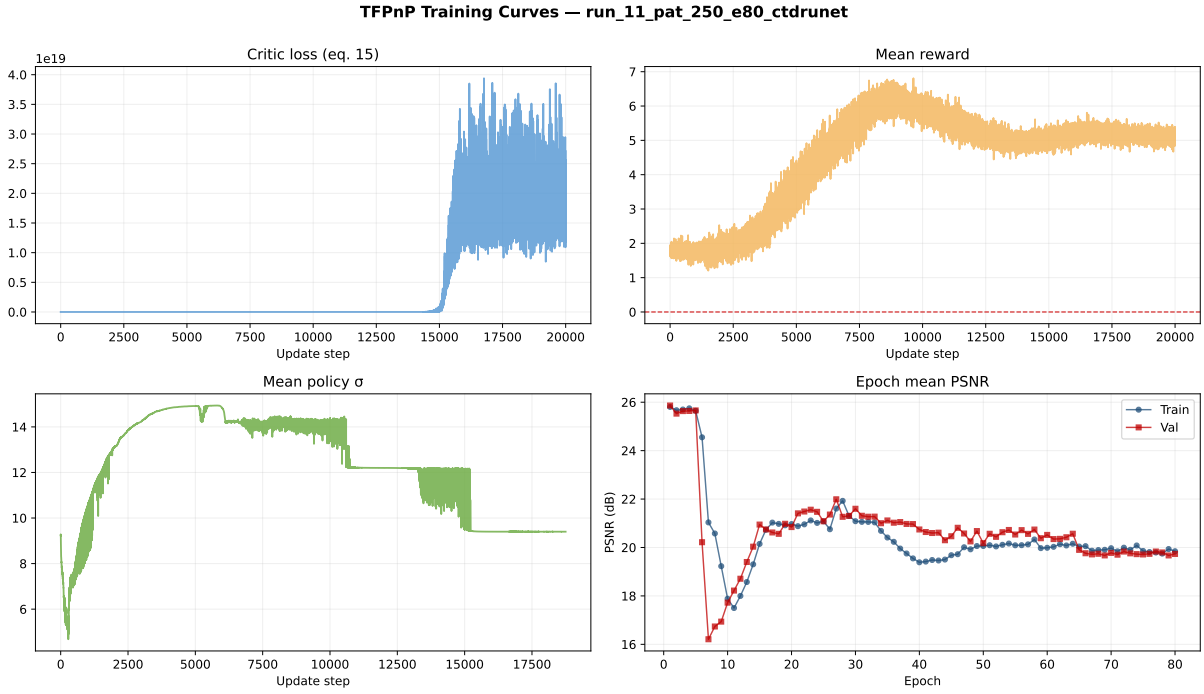

Figure 4.8: Training curves for `run_11`.

4.7 Reward shaping ablations

Four variant runs were carried out to test whether alternative reward signals could improve metric-specific performance, motivated by the finding of Breger et al. [2025] that PSNR correlates poorly with perceived image quality in medical imaging, raising the question of whether an RL policy rewarded on different metrics would produce better reconstructions than one trained on PSNR alone. The study is an extension undertaken in addition to the reproduction, rather than a component of Wei et al. [2022]'s original methodology, and results are summarised in Table 4.3.

The reward coefficients compensate for the difference in numeric scale between metrics, as a

typical per-step ΔPSNR is on the order of 0.1–1.0, whereas ΔSSIM and ΔHaarPSI are on the order of 0.001–0.01, so without scaling would be roughly 100× smaller than the PSNR term and contribute negligibly to the policy gradient. The mixed rewards (`run_07`, `run_08`) therefore use $\alpha = 5$ to bring the perceptual term to a comparable order of magnitude as ΔPSNR, and the pure-replacement rewards (`run_09`, `run_10`) use $\alpha = 50$ so the overall reward magnitude remains comparable to `run_04`'s pure-PSNR reward.

The SSIM-only reward (`run_09`) achieves the highest PSNR and HaarPSI among all runs, and the mixed PSNR/SSIM reward (`run_07`) wins on SSIM specifically. In contrast, both HaarPSI-based rewards (`run_08` PSNR/HaarPSI, `run_10` HaarPSI only) underperform across all metrics. Given only a single training seed is used per configuration, some of this variation likely reflects RL training stochasticity rather than the differences between reward functions, so the PSNR reward chosen for `run_04` remains a defensible default for direct comparison to the paper.

Table 4.3: Reward-shaping ablation at 5% noise. Best values in **bold**.

Run	Reward function	PSNR	SSIM	HaarPSI
`run_04`	PSNR (main)	27.27	0.691	0.485
`run_07`	PSNR + SSIM	27.18	**0.707**	0.481
`run_08`	PSNR + HaarPSI	26.47	0.640	0.437
`run_09`	SSIM only	**27.32**	0.695	**0.489**
`run_10`	HaarPSI only	26.76	0.658	0.460

Chapter 5

Discussion

5.1 Contextualising the TFPnP–FBPConvNet comparison

The reproduction confirms Wei et al. [2022]'s central claims, as the learned policy improves on hand-calibrated hyperparameters by +2.96 dB at 5% noise (Section 4.1), and TFPnP degrades gracefully across the noise range (Section 4.2). The one direct disagreement with the paper is the FBPConvNet–TFPnP ranking at 5% noise, which reverses, and can be explained by two factors.

The first is an in-domain training asymmetry. In Wei et al. [2022], both the denoiser (BSD400) and policy (PASCAL VOC) are trained on natural images, and FBPconv's flat PSNRs across noise levels are consistent with out-of-domain training. In this reproduction, FBPConvNet is fully in-domain (LIDC), while TFPnP is only partially so, with its policy being LIDC-trained but its denoiser remaining as the natural-image DRUNet. At low noise, the fully in-domain method extracts fine anatomical detail from the FBP reconstruction more effectively than the natural-image denoiser can.

The second is architectural, as TFPnP overtakes FBPConvNet at 10% noise on all three metrics (Figure 4.2). Part of this is due to a training-distribution effect, as FBPConvNet was trained at fixed 5% noise while TFPnP saw the mixed regime, but the crossover on the perceptual metrics is unlikely to be explained by that alone. Repeated denoising passes appear to remove sinogram noise progressively in a way that a single forward pass is unable to, highlighting the iterative advantage of TFPnP despite the in-domain advantage of FBPConv.

5.2 Does an in-domain denoiser close the gap?

If FBPConvNet's advantage at low noise comes from in-domain training, a natural extension is to give TFPnP the same advantage by training the denoiser on CT. The CT-DRUNet extension (Section 3.6) explores this, providing one robust finding and one negative result.

In the Fixed PnP-ADMM setting, the CT-trained DRUNet outperforms the natural-image DRUNet at every noise level, with the advantage growing under noise (Table 4.2). Because each denoiser is evaluated at its own calibrated optimum, the denoising strengths are matched,

and the improvement is linked to in-domain training, where a denoiser that has seen CT statistics is able to separate signal from iteration noise more reliably. This result stands independently of the full TFPnP pipeline.

The negative result is that retraining the TFPnP policy against the CT-DRUNet (`run_11`) with a widened σ action range failed due to the critic loss diverging and the policy σ output collapsing (Section 4.6). At the time this was attributed to either the widened action range enlarging the space of ADMM states the critic must estimate and our value network omitting the weight normalisation and TReLU that Table 1 of Wei et al. [2022] specifies for stability, however a subsequent diagnosis identified a third cause that is set discussed in Section 5.5, together with the corrections it motivated.

Taken together, the in-domain denoiser advantage is real and reproducible in the Fixed PnP-ADMM setting, but was not translated into the RL-based TFPnP pipeline within the project timeline.

5.3 On the choice of evaluation metrics

PSNR is the primary metric throughout, chosen for direct comparison to Wei et al. [2022]. It captures only pixel-wise mean squared error, weighting every pixel equally, so a blurred reconstruction that smooths away texture can score identically to one that preserves texture but adds slight noise. SSIM partially addresses this by comparing local luminance, contrast, and structure, which HaarPSI extends through a Haar wavelet decomposition and has been shown to correlate more strongly with radiological assessment [Breger et al., 2025]. Reporting all three guards against over-reading a single pixel-wise metric.

5.4 On direct comparison with Wei et al. [2022]

The absolute PSNR values here cannot be compared number-for-number with the paper's, for four reasons. The PSNR data-range convention differs (LION's per-image `data_range` $= x_{\mathrm{gt}}.\max()$ in μ units versus the paper's likely 1.0 on $[0, 1]$ images), which alone accounts for a $\sim 5\,\mathrm{dB}$ shift for the same mean squared error. The test sets (50 COVID-19 slices versus 389 LIDC slices), noise conventions (scaling detail), and several implementation choices also differ, consolidated in Table 3.1. The most significant change is the μ range ($[10, 100]$ versus $[0.01, 1]$), which is a direct consequence of the z-step gradient normalisation rather than a free hyperparameter, since LION's unnormalised adjoint is $\sim 100\times$ larger than TorchRadon's.

Two limitations of the comparison itself are also worth mentioning. First, only three of the paper's six baselines are reproduced in this implementation, as RED-CNN and RPGD are absent from LION, and LPD would require training beyond the timeline. Second, compute constraints meant that every run used one random seed, so the reported numbers reflect one RL trajectory rather than an ensemble. The reproduction's central claims are therefore best read as within-experiment findings, with TFPnP outperforming Fixed PnP-ADMM by $+2.96\,\mathrm{dB}$ at 5% noise, and the FBPConvNet ranking inverting with noise, rather than as absolute reproductions.

5.5 Corrections identified but not yet validated

Preparing the codebase and accompanying test suite revealed several issues that were diagnosed and corrected in code but could not be validated against fresh training runs before a CSD3 cluster outage ended the project's compute window. They are recorded here for transparency and as the immediate starting point for future work.

Re-diagnosis of the `run_11` failure. The CT-DRUNet's apparent optimum at $\sigma \approx 8$, against the natural-image DRUNet's $\sigma \approx 1.5$ (Section 4.4) was originally read as evidence that in-domain training shifts the denoiser's regularisation scale. However, closer analysis of the training code shows that this is largely due to the training script applying the noise level and the network's noise-level input channel on two different scales due to an unchanged parameter. Converting both optima to a common noise scale places them at essentially the same denoising strength, reframing `run_11`, as the σ action range was widened to $[1, 15]$ to reach an optimum that, with a consistently calibrated denoiser, would have sat within the original $[1, 5]$ range. The widened range enlarged the RL state space and destabilised the un-normalised critic for no necessary reason. The corrected training script now uses a single noise scale, making the CT-DRUNet a true drop-in for the natural-image denoiser, with the intended rescue now being to retrain with `run_04`'s original action ranges, rather than the widened ranges of `run_11` and `run_12`. This does not affect the Fixed PnP-ADMM result of Table 4.2, where each denoiser is used at its own measured optimum.

Prepared but unrun experiments. A mixed-noise FBPConvNet training script was prepared to address the fixed-5% limitation of the baseline (Section 3.5), which is the main finding in the cross-noise comparison of Section 4.2. The script was ready to submit when the outage began but could not be completed.

5.6 Broader implications

This reproduction reveals two broader observations, the first being that, Wei et al. [2022]'s fair-comparison methodology, in which all learned methods are evaluated with out-of-domain training data, can obscure the capabilities of certain methods on a specific medical domain, as allowing FBPConvNet to train in-domain substantially changes its standing relative to TFPnP. This does not invalidate the paper's contribution, but it argues for comparing PnP methods against domain-adapted baselines. Second, TFPnP's tuning-free claim is accurate at inference but not during development, as the reward-shaping ablation (Section 4.7) shows meaningful differences across reward configurations, and the `run_11` experience shows that swapping the denoiser requires care with calibration and action ranges, highlighting that tuning is incredibly demanding during development.

Chapter 6

Conclusion

This project reproduced the Tuning-Free Plug-and-Play method of Wei et al. [2022] within the LION toolbox [Biguri et al., 2024] for 30-view parallel-beam sparse-view CT on LIDC-IDRI. The reproduction implements the RL policy-learning procedure, the ADMM environment, and the core baselines, and reproduces the paper's central claim that TFPnP improves by +2.96 dB over hand-calibrated PnP-ADMM, achieving 27.27 dB at 5% noise. The result is delivered as a documented, tested Python package integrated with LION, together with an upstream contribution (pull request #184) fixing a bug in LION's FBPConvNet integration.

The evaluation surfaces two findings the paper does not report. First, giving FBPConvNet in-domain training reverses its ranking against TFPnP at low noise (+1.31 dB FBPConvNet advantage at 5%), while TFPnP retains its advantage at 10% across all three metrics. Second, a CT-specific DRUNet trained on LIDC improves Fixed PnP-ADMM by +0.71–+1.98 dB, with the advantage growing under noise. Translating this into a fully retrained TFPnP policy encountered instability, but a diagnosis (Section 5.5) traced this partially to a calibration artefact in the CT-DRUNet training, which the released code corrects but the compute outage prevented from re-testing.

Immediate follow-up work is well defined by the prepared-but-unvalidated corrections of Section 5.5. The main priorities will be retraining the CT-DRUNet policy against the recalibrated denoiser at the original action ranges and running the prepared mixed-noise FBPConvNet baseline. Beyond these, adding the RED-CNN, LPD, and RPGD baselines absent from LION would complete the comparison list, and extending the infrastructure to other plug-and-play algorithms and to fan or cone-beam geometries via LION's tomosipo backend would explore the generalisation of TFPnP. A significant finding is that plug-and-play methods demand careful denoiser calibration when the imaging domain shifts, and that reproducibility studies remain a valuable mechanism for surfacing choices that primary papers leave underspecified.

Appendix A

Supplementary Material

A.1 Compute infrastructure

All experiments were run on the Cambridge Service for Data Driven Discovery (CSD3) HPC cluster, using GPUs on the `ampere` partition. GPU access was allocated through a single `MPHIL-DIS-SL2-GPU` SLURM account, which imposed a cap on the number of ablation and rescue runs that could fit within the project timeline. All jobs were managed through SLURM batch scripts, against a project-specific conda environment (`mphil_ct`, Python 3.11) providing the LION toolbox, PyTorch, tomosipo, and supporting packages.

Total compute across all experiments was approximately 290 GPU-hours: ∼28 h per TFPnP training run across ten completed runs, plus ∼2.5 h for CT-DRUNet training, ∼0.5 h for FBP-ConvNet training, and the remainder split between hyperparameter sweeps and evaluation. A batched SLURM evaluation script (`slurm/full_eval_batch.sh`) reproduces all reported metrics from saved checkpoints in a single job.

A.2 Run matrix

Table A.1 summarises all eleven TFPnP training runs carried out during the project. `run_04` is the canonical reproduction run reported throughout Chapter 4, with all other runs varying one aspect at a time relative to it.

Table A.1: Summary of TFPnP training runs. Runs 04–12 use 250 LIDC patients, 80 epochs, and the paper's default hyperparameters unless noted.

Run	Change vs. `run_04`	Rationale
`run_01`	20 patients, 20 epochs	Notebook-development sanity check that the pipeline runs and RL training converges
`run_02`	100 patients, 50 epochs	Scaling test to confirm behaviour holds beyond the 20-patient setting
`run_03`	Full LIDC split (3300 slices), 5 epochs	Test whether more patients with fewer epochs improves policy quality at similar total training budget
`run_04`	Main run: natural-image DRUNet, PSNR reward, $\sigma \in [1, 5]$, $\mu \in [10, 100]$	Direct reproduction of Wei et al. [2022]
`run_05`	σ floor lowered from 1.0 to 0.5	Test whether weaker denoising lets the policy fine-tune reconstructions in later ADMM iterations
`run_06`	$m = 3$, $N = 10$ (total ADMM steps unchanged)	Test sensitivity to the paper's $m = 5$, $N = 6$ inner-outer split
`run_07`	Mixed PSNR/SSIM reward	Test whether a joint reward improves perceptual quality (Section 4.7)
`run_08`	Mixed PSNR/HaarPSI reward	Test HaarPSI-based reward against Breger et al. [2025]'s motivation
`run_09`	SSIM-only reward	Isolate the structural-similarity signal
`run_10`	HaarPSI-only reward	Isolate the perceptual-similarity signal
`run_11`	CT-DRUNet denoiser; $\sigma \in [1, 15]$; $\mu \in [5, 50]$	Test in-domain denoiser inside the full RL pipeline
`run_12`	CT-DRUNet; narrower action range; longer π_2 warmup	Rescue attempt for `run_11`, prepared but not submitted before the CSD3 outage; superseded by the recalibrated-denoiser approach of Section 5.5

A.3 Repository and artefacts

The full source for the reproduction is hosted at `https://gitlab.developers.cam.ac.uk/phy/data-intensive-science-mphil/assessments/projects/eaz21`.

The repository is organised as follows:

- `src/ct_tfpnp/` contains the Python package (policy network, value network, ADMM environment, denoiser wrappers, training loop, and baseline reconstructors)
- `scripts/` contains the training and evaluation scripts (`train_tfpnp.py`, `train_drunet_ct.py`, `train_fbpconvnet.py`, and the plotting and evaluation utilities).
- `slurm/` contains one SLURM launcher per training run (`train_tfpnp_02.sh` through `train_tfpnp_12.sh`) plus the baseline training and batch evaluation scripts.
- `notebooks/` contains the development narrative NB01–NB12, covering tomosipo and CT

physics fundamentals, LION exploration, PnP-ADMM development, TFPnP paper analysis, policy and environment prototyping, the full training pipeline, baselines, and the CT denoiser extension.

- `data/` contains a small LIDC subset for local smoke tests, with the full dataset loaded through LION's dataloader on CSD3.
- `results/` contains the metrics CSVs, but per-run checkpoints are on CSD3 due to size.
- `figures/` contains the plots for each run and each notebook chapter.
- `report/` contains the LaTeX source, bibliography, and compiled PDF for this report.

Model checkpoints are too large to commit to the repository and live on CSD3, available on request:

- Natural-image DRUNet weights, pretrained from Zhang et al. [2021] (`results/baselines/drunet_gray.p`
- CT-DRUNet best checkpoint (`results/baselines/drunet_ct/drunet_ct_best.pth`).
- FBPConvNet checkpoint (`results/baselines/fbpconvnet/fbpconvnet_5pct_e80.pth`).
- TFPnP canonical checkpoint (`results/learned/run_04_pat_250_e80/best.pt`).

The upstream LION pull request (#184) contributing a fix for the FBPConvNet integration is available at `https://github.com/CambridgeCIA/LION/pull/184`.

A.4 AI/LLM usage

Large language models (primarily Gemini and Claude) were used to support this project in the following areas:

- **Plotting boilerplate.** Matplotlib scaffolds for training curves, reconstruction galleries, and sensitivity sweeps.
- **Library usage.** Working out calling conventions for alignment with LION.
- **Debugging.** The CT-DRUNet calibration mismatch (Section 3.6), the `run_11` critic explosion (Section 4.6), and unit-scale mismatches between LION and TorchRadon (Section 3.4).
- **Docstrings and documentation.** Function-level docstrings for `ct_tfpnp` and generation of documentation using Sphinx.
- **Report review.** Redundancy checks, cross-referencing and typo fixes.
- **Cross-checking against the paper.** Comparing implementation choices with Wei et al. [2022] and consolidating differences into Table 3.1.

All model output was reviewed and, where used, edited before being committed to the repository or incorporated into this report. No model output was accepted without verification against the source paper, the LION toolbox, or empirical results from the reproduction itself. The core experimental design, hyperparameter choices, framing of the findings, and interpretation of the results are my own.

Bibliography

Samuel G. Armato, Geoffrey McLennan, Luc Bidaut, Michael F. McNitt-Gray, Charles R. Meyer, Anthony P. Reeves, Binsheng Zhao, Denise R. Aberle, Claudia I. Henschke, Eric A. Hoffman, et al. The Lung Image Database Consortium (LIDC) and Image Database Resource Initiative (IDRI): A completed reference database of lung nodules on CT scans. *Medical Physics*, 38(2): 915–931, 2011.

Ander Biguri, Subhadip Mukherjee, Xuzhi Zhao, Xi Liu, Xinyi Wang, Rui Yang, Yi Du, Yahui Peng, Mikael Brudfors, Mark Graham, Hyungon Ryu, Oliver Kutter, Andreas Hauptmann, Mustafa Al-Rubaye, Miika T. Nieminen, Mikael A. K. Brix, Austin Yunker, Rajkumar Kettimuthu, John C. Roeske, Sasidhar Alavala, Subrahmanyam Gorthi, and Carola-Bibiane Schönlieb. Advancing the frontiers of deep learning for low-dose 3d cone-beam ct reconstruction. *IEEE Open Journal of Signal Processing*, 6:942–949, 2025. doi: 10.1109/OJSP.2025.3593189.

Ander Biguri et al. LION: Learned Iterative Optimization Networks, 2024. URL `https://github.com/CambridgeCIA/LION`. Cambridge Image Analysis Group.

Anna Breger, Ander Biguri, Malena Sabaté Landman, Ian Selby, Nicole Amberg, Elisabeth Brunner, Janek Gröhl, Sepideh Hatamikia, Clemens Karner, Lipeng Ning, et al. A study of why we need to reassess full reference image quality assessment with medical images. *Journal of Imaging Informatics in Medicine*, 38(6):3444–3469, 2025.

Allard A. Hendriksen, Daniël M. Pelt, and K. Joost Batenburg. Tomosipo: fast, flexible, and convenient 3D tomography for complex scanning geometries in Python. *Optics Express*, 29 (24):40494–40513, 2021.

Kyong Hwan Jin, Michael T. McCann, Emmanuel Froustey, and Michael Unser. Deep convolutional neural network for inverse problems in imaging. *IEEE Transactions on Image Processing*, 26(9):4509–4522, 2017. doi: 10.1109/TIP.2017.2713099.

Sergey Kastryulin, Jamil Zakirov, Denis Prokopenko, and Dmitry V. Dylov. Pytorch image quality: Metrics for image quality assessment, 2022. URL `https://arxiv.org/abs/2208.14818`.

Jun Ma, Yixin Wang, Xingle An, Cheng Ge, Ziqi Yu, Jianan Chen, Qiongjie Zhu, Guoqiang Dong, Jian He, Zhiqiang He, Tianjia Cao, Yuntao Zhu, Ziwei Nie, and Xiaoping Yang. Toward data-efficient learning: A benchmark for covid-19 ct lung and infection segmentation. *Medical*

Physics, 48(3):1197–1210, 2021. doi: https://doi.org/10.1002/mp.14676. URL `https://aapm.onlinelibrary.wiley.com/doi/abs/10.1002/mp.14676`.

Rafael Reisenhofer, Sebastian Bosse, Gitta Kutyniok, and Thomas Wiegand. A haar wavelet-based perceptual similarity index for image quality assessment. In *Signal Processing: Image Communication*, volume 61, pages 33–43, 2018.

Matteo Ronchetti. Torchradon: Fast differentiable routines for computed tomography. *ArXiv*, abs/2009.14788, 2020. URL `https://api.semanticscholar.org/CorpusID:222067122`.

Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4): 600–612, 2004.

Kaixuan Wei, Angelica Aviles-Rivero, Jingwei Liang, Ying Fu, Hua Huang, and Carola-Bibiane Schönlieb. TFPnP: Tuning-free plug-and-play proximal algorithms with applications to inverse imaging problems. *Journal of Machine Learning Research*, 23(16):1–48, 2022.

Kai Zhang, Yawei Li, Wangmeng Zuo, Lei Zhang, Luc Van Gool, and Radu Timofte. Plug-and-play image restoration with deep denoiser prior. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):6360–6376, 2021.